The Geometry and Metric Tensor of a Trained Multilayer Perceptron

James Squires

May 2026

1 Introduction

These notes are meant to briefly characterize the differential geometry of a neural network. While parameter space and the information geometry of neural networks are heavily explored, the differential geometry of the manifold given by a trained neural network mapping $f : \mathbb{R}^m \to \mathbb{R}^n$ remain criminally underexplored despite the immense value that this would provide.

2 Foundations

2.1 Hypothesis Making

In order to build a predictive model, first we have to assume that the model we're attempting to predict has a particular distribution/shape. Doing so, we are then able to fine-tune this distribution to best match the actual data. These assumptions are typically informed by the specific problem.

Formally, suppose there is a relationship f between $\mathbf{x} \in \mathbb{R}^m$ and $y \in \mathbb{R}$. Then we write this as the function:

$$f : \mathbb{R}^m \to \mathbb{R}^m \tag{1}$$

$$\mathbf{x} \mapsto f(\mathbf{x})$$

In order to predict such a relationship, we define a *hypothesis class* of possible functions on the parameters $\theta \in \mathbb{R}^k$:

$$\mathcal{H}(\hat{f}, \theta) = \{\hat{f}_\theta | \theta \in \mathbb{R}^k\} \tag{2}$$

2.2 Loss Function

Once we define the hypothesis class, we then define the loss function $\mathcal{L}(\hat{f}, \theta) : \mathbb{R}^k \to \mathbb{R}$. For our purposes, we will use the RSS loss-function.

Definition 2.1 (Residual Sum of Squares (RSS))**.** The residual sum of squares is the sum of the square residuals...

$$RSS = \sum_{i=1}^{n} (\hat{f}(\mathbf{x}_i) - y_i)^2 \tag{3}$$

Once this loss-function has been defined, we minimize the loss function to obtain our *best-fit* model:

$$\nabla \mathcal{L}(\theta) = \mathbf{0} \tag{4}$$

Most loss functions are non-convex, thus finding a global minimum is not guaranteed.

For the sake of studying a neural network that has already been trained, we will assume $\nabla \mathcal{L}(\theta) \leq \epsilon$ for the rest of this paper.

3 Affine Geometry

3.1 Review of Linear Algebra Geometry

3.1.1 Hyperplanes

Using the identity[1]:

$$\mathbf{w} \cdot \mathbf{x} = ||\mathbf{w}||||\mathbf{x}||cos(\angle(\mathbf{x}, \mathbf{w})) \tag{5}$$

where $\angle(\mathbf{x}, \mathbf{w})$ denotes the angle between $\mathbf{x}$ and $\mathbf{w}$, we know that any vector orthogonal to $\mathbf{w}$ satisfies $\mathbf{v} \cdot \mathbf{x} = 0$. Thus, if we want to describe a plane of vectors $\mathbf{x} \in \mathbb{R}^m$, we simply define the set of vectors as all the vectors orthogonal to the normal vector $\mathbf{v}$:

$$\mathbf{w} \cdot \mathbf{x} = 0 \tag{6}$$

Now if we imagine data-points in $\mathbb{R}^m$, then finding the hyperplane that has the least distance between the different data-points would likely require a *translation*. This is done by adding some constant to the hyperplane:

$$\mathbf{w} \cdot \mathbf{x} = b \tag{7}$$

Typically we call the constant $b \in \mathbb{R}$ the *bias*. This translates the plane a magnitude of $\frac{b}{||w||}$ in the direction of $\mathbf{w}$.

[1] We will bold vectors in $\mathbb{R}^m$ where $m > 1$, and not bold scalar components.

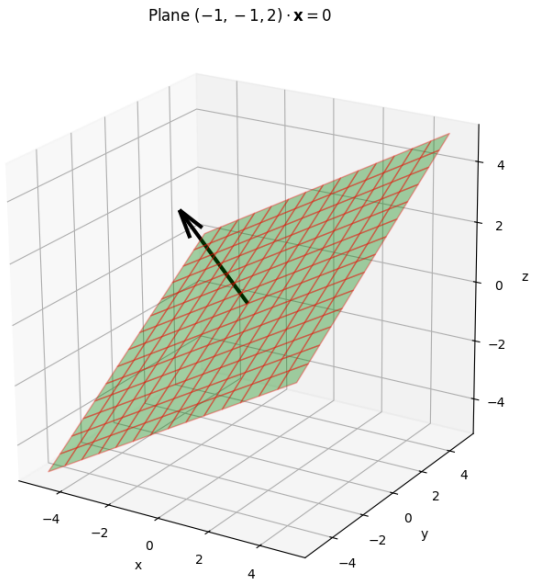

Figure 1: A hyperplane through the origin.

3.1.2 Affine Transformations

Given a vector $\mathbf{x} \in \mathbb{R}^m$, we can linearly transform this vector with a matrix $A \in \mathbb{R}^n \times \mathbb{R}^m$. We can imagine every arbitrary vector $\mathbf{x} \in \mathbb{R}^m$ undergoing such a transformation, so A can be thought of as a transformation acting on the vector space itself:

$$A : \mathbb{R}^m \to \mathbb{R}^n \tag{8}$$

$$\mathbf{w} \mapsto A\mathbf{w}$$

Sometimes we also want to discuss a translation of coordinates. Doing so gives us a more general transformation called an affine transformation.

Definition 3.1 (Affine Transformation)**.** An *affine transformation* is a linear transformation that also introduces a translation $\mathbf{b} \in \mathbb{R}^n$:

$$A : \mathbb{R}^m \to \mathbb{R}^n \tag{9}$$

$$\mathbf{x} \mapsto W\mathbf{x} + \mathbf{b}$$

$\mathbf{b}$ is also referred to as the *bias*.

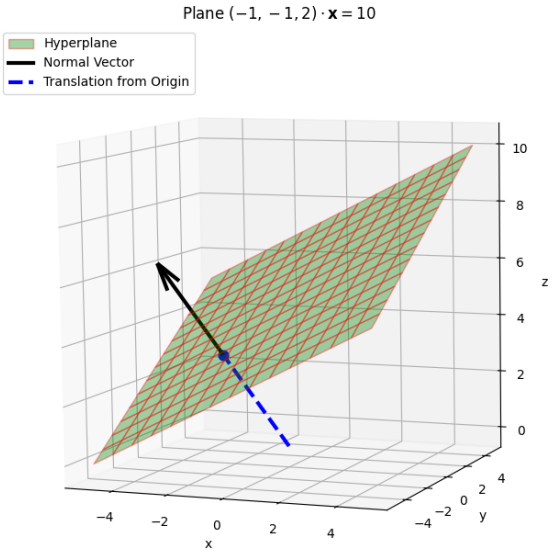

Figure 2: A hyperplane through the $p = \frac{b}{||\mathbf{v}||^2}\mathbf{v}$.

Of course, an affine transformation gives us a system of affine hyperplanes[2]:

$$W\mathbf{x} + \mathbf{b} = \begin{pmatrix} \mathbf{w}_1 \cdot \mathbf{x} + b_1 \\ \mathbf{w}_2 \cdot \mathbf{x} + b_1 \\ \vdots \\ \mathbf{w}_n \cdot \mathbf{x} + b_1 \end{pmatrix} \tag{10}$$

Any vector $\mathbf{x}$ that lies on every one of these affine planes is considered a solution.

Remark 3.1. For those curious, the set of all invertible affine transformations forms a group. Full-rank affine transformations are not guaranteed in a neural network though.

Just for fun, we will show how affine transformations form a group. Skip ahead to section 4 if this is uninteresting.

In order to describe the group of affine transformations $AG(m, \mathbb{R})$, we will describe such transformations inside of $GL(m+1, \mathbb{R})$ by rewriting $W\mathbf{x}+\mathbf{b}$ in block

[2] $\mathbf{w}_i$ denotes a row of W.

matrix form[3]:

$$AG(m,\mathbb{R}) := \begin{pmatrix} W & \mathbf{b} \\ 0 & 1 \end{pmatrix},\ \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{pmatrix} \in \mathbb{R}^{m+1}$$

Theorem 3.1. *$AG(m,\mathbb{R})$ forms a group.*

Proof. $AG(m,\mathbb{R})$ inherits identity and associativity. We just show closure and invertibility. Matrix multiplication gives the block matrix:

$$\begin{pmatrix} A & \mathbf{a} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} B & \mathbf{b} \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} AB & A\mathbf{b}+\mathbf{a} \\ 0 & 1 \end{pmatrix} \tag{11}$$

Now it is obvious that closure is satisfied.

Likewise, the inverse is given as follows:

$$\begin{pmatrix} A & \mathbf{a} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} A & \mathbf{a} \\ 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} A & \mathbf{a} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} A^{-1} & -A^{-1}a \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} I_n & \mathbf{0} \\ 0 & 1 \end{pmatrix} = I_{n+1} \tag{12}$$

□

4 Neural Networks

Definition 4.1 (Sigmoid Function)**.** We will use the sigmoid function (see Fig. 3):

$$\sigma(x_i) = \frac{1}{1+e^{-x_i}} \tag{13}$$

We will now define what an Artificial Neural Network is.

Definition 4.2 (Layer)**.** The ith *layer* of a neural network is an affine transformation followed by a sigmoid function on each component[4]:

$$H^{(i)} = \sigma^{(i)} \circ \mathbf{W}^{(i)} : \mathbb{R}^{m_{i-1}} \to \mathbb{R}^{m_i} \tag{14}$$
$$\mathbf{x}^{(i-1)} \mapsto \sigma^{(i)}(W^{(i)}\mathbf{x}^{(i-1)} + \mathbf{b}^{(i)})$$

Remark 4.1. $\sigma^{(i)}$ is an activation function of the ith layer. Many different activation functions can be used.

An Artificial Neural Network (ANN), also colloquially called a Multilayer Perceptron (MLP) is defined as follows:

[3] $AG(m,\mathbb{R}) \hookrightarrow GL(m+1,\mathbb{R})$

[4] We're indexing the dimension of the output space of the ith layer by m_i.

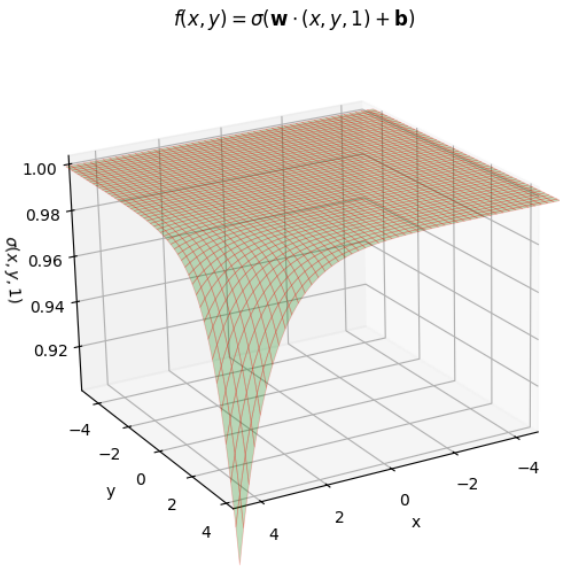

Figure 3: $\sigma(x, y, 1)$ with row j of $W^{(i)}$ and $b^{(i)}$.

Definition 4.3 (Multilayer Perceptrons)**.** An MLP is the function mapping:

$$f : \mathbb{R}^m \times \mathbb{R}^\theta \to \mathbb{R}^n \tag{15}$$
$$\mathbf{x} \mapsto H^{(L)} \circ ... \circ H^{(1)}(\mathbf{x})$$

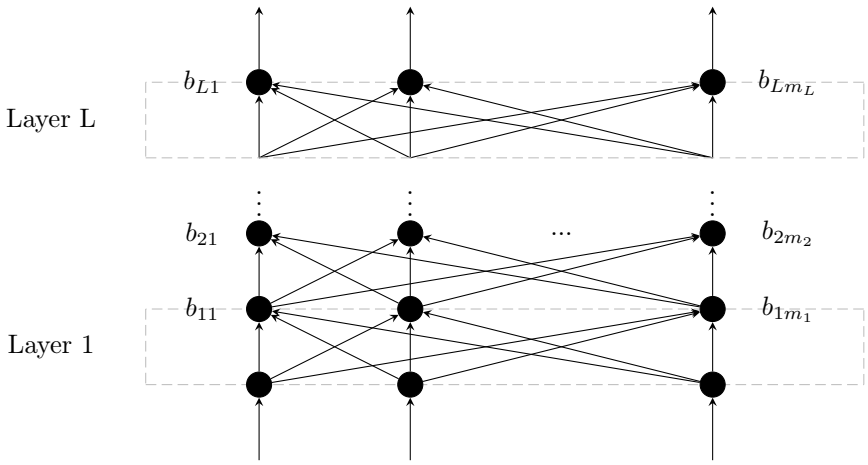

Figure 4: The Biases of Layer i Associated with that Layer's jth neuron, given as b_{ij}.

5 Differential Geometry

We would like to study the Riemannian geometry of the MLP. In order to do so, we first define the metric tensor. We will use the Jacobian matrix definition.

Definition 5.1 (Jacobian Matrix)**.** The *Jacobian* of a function

$$f_\theta : \mathbb{R}^m \to \mathbb{R}^n$$

$$(x_1, ..., x_m) \mapsto (f_1(x_1, ..., x_m), \dots, f_n(x_1, \dots, x_m))$$

is given by:

$$J_f = \begin{pmatrix} \nabla^T f_1 \\ \nabla^T f_2 \\ \vdots \\ \nabla^T f_n \end{pmatrix} \tag{16}$$

wheere $\nabla^T f_i$ is[5] is a row-vector.

Definition 5.2 (Metric Tensor)**.** The *metric tensor* of $f : \mathbb{R}^m \to \mathbb{R}^n$ is given by $g = J_f^T J_f$.

Remark 5.1. Picking out component g_{ij} gives:

$$g_{ij} = \begin{pmatrix} \frac{\partial f^1}{\partial x^i} & \dots & \frac{\partial f^n}{\partial x^i} \end{pmatrix} \begin{pmatrix} \frac{\partial f^1}{\partial x^j} & \dots & \frac{\partial f^n}{\partial x^j} \end{pmatrix}^T = \sum_{k=1}^{n} \frac{\partial f^k}{\partial x^i} \frac{\partial f^k}{\partial x^j}$$

[5]We're going to drop the θ subscript when convenient. Take $f = f_\theta$.

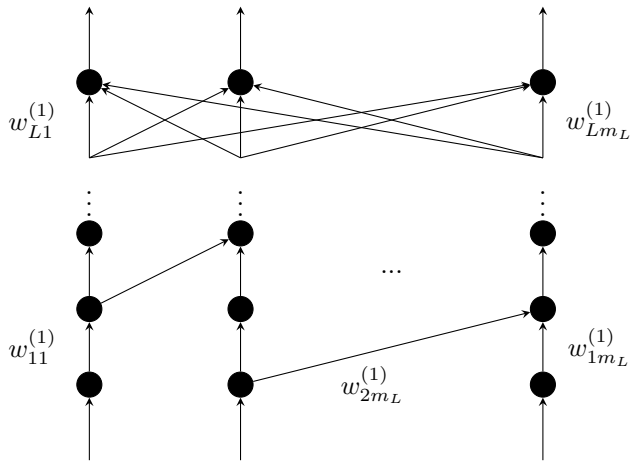

Figure 5: Weights given as connections from the jth neuron in layer $i-1$ to the kth neuron in layer i, denoted $w_{jk}^{(i)}$. Some of the connections are dropped for clarity.

5.1 Differential Geometry of an MLP

Taking the Jacobian of equation 15, we have[6]:

$$J_{f_\theta} = J_{H^{(L)}} ... J_{H^{(1)}} \tag{17}$$

Picking out a particular index according to Equation 16, we have as components of each $J_{H^{(l)}}$:

$$J_{ij} = \frac{\partial H_i^{(l)}}{\partial x_j} = \frac{\partial(\sigma(\mathbf{w_i} \cdot \mathbf{x} + b_i))}{\partial x_j} \tag{18}$$

We drop the superscript (l) notation for this derivation, but keep in mind the bias $\mathbf{b} = \mathbf{b}^{(l)}$, and likewise for the input $\mathbf{x}^{(l)}$, weight matrix $W^{(l)}$, and the activation function $\sigma^{(l)}$.

When we write b_i, this is the ith row of the bias $\mathbf{b}$. Likewise $\mathbf{w}_i$ is the ith row of W.

Letting $z_i = \mathbf{w_i} \cdot \mathbf{x} + b_i$, chain-rule gives:

$$\frac{\partial(\sigma(z_i))}{\partial x_j} = \frac{\partial \sigma(z_i)}{\partial z_i} \frac{\partial z_i}{\partial x_j} \tag{19}$$

Expanding out $\frac{\partial z_i}{\partial x_j}$, we have[7]:

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j} \left(\sum_{k=1}^{m} (w_{ik} x_k) + b_i \right) = w_{ij} \tag{20}$$

[6] By chain-rule.

[7] Where k indexes the m columns. Remember that W is an $n \times m$ matrix.

Therefore:

$$J_{ij} = \frac{\partial \sigma(z_i)}{\partial z_i} w_{ij} \tag{21}$$

Remarkably, this gives us the Jacobian:

$$J_{H^{(l)}} = \begin{pmatrix} \frac{\partial \sigma(z_1)}{\partial z_1} w_{11} \dots \frac{\partial \sigma(z_1)}{\partial z_1} w_{1m} \\ \vdots \qquad\qquad\qquad \vdots \\ \frac{\partial \sigma(z_n)}{\partial z_n} w_{n1} \dots \frac{\partial \sigma(z_n)}{\partial z_n} w_{nm} \end{pmatrix} \tag{22}$$

Simplifying this by recognizing[8] we can factor out the diagonol matrix with diagonols given by $D^{(l)} = (\frac{\partial \sigma(z_1)}{\partial z_1}, \dots, \frac{\partial \sigma(z_n)}{\partial z_n})$, we have:

$$J_{H^{(l)}} = D^{(l)} W^{(l)} \tag{23}$$

Thus, the Jacobian J_{f_θ} is given as [9]:

$$J_{f_\theta} = D^{(L)} W^{(L)} \dots D^{(1)} W^{(1)} = \prod_{l=0}^{L-1} D^{(L-l)} W^{(L-l)} \tag{24}$$

Applying Definition 5.2:

$$g_{f_\theta} = \prod_{l=1}^{L} (W^{(l)})^T D^{(l)} \prod_{l=0}^{L-1} (D^{(L-l)} W^{(L-l)}) \tag{25}$$

This gives the activation functions a clean role in terms of the intrinsic geometry of a neural network. In each of the L layers, input space is linearly transformed by the weight matrix. This is followed by the *anisotropic*[10] scaling of the linearly transformed input space.

By deriving the metric, we have characterized the (likely) non-Euclidean geometry of an MLP. I'd argue the real value comes from the Second Fundamental Form and principal curvatures of the MLP though, as high curvature regions are extremely informative. A high curvature region tells us that a perturbation in input space of a particular region result in drastically different outcomes. This isn't so much mechanistic interpretability as it is raw geometric understanding. However, the computations for this are quite difficult, so we will leave this exposition as is.

[8] Another reminder that we've dropped the superscripts (l) on all our parameters.
[9] Notice the Π convention we're using: $L \to L-1 \to ... \to 1$.
[10] Non-uniform scaling.